



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT,Vellore.



Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms
2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively
3. To synthesize efficient algorithms in various engineering design situations



BCSE204L- Design and Analysis of Algorithms- Course outcome

1. Apply the mathematical tools to analyse and derive the running time of the algorithms
2. Demonstrate the major algorithm design paradigms.
3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.
4. Articulating Randomized Algorithms.
5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	Design Paradigms: Greedy, Divide and Conquer Techniques	6 hours
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.		
Module:2	Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques	10 hours
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- Backtracking: N-Queens problem, Subset Sum, <u>Graph Coloring</u> - Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	String Matching Algorithms	5 hours
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	Graph Algorithms	6 hours
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	Geometric Algorithms	4 hours
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	Randomized algorithms	5 hours
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	Classes of Complexity and Approximation Algorithms	7 hours
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	Contemporary Issues	2 hours



BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, Éva Tardos, Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press, 1995 (Online Print – 2013)
3. Ravindra K. Ahuja, Thomas L. Magnanti and James B. Orlin, “ Network Flows: Theory, Algorithms and Applications”, Pearson Education, 2014.



Module 2: Design Paradigms: Dynamic Programming-Backtracking- Branch and Bound(10 Hours)

- **Dynamic Programming**
 - Matrix chain Multiplication
 - Assembly line Scheduling
 - Longest Common Subsequence
 - 0/1 Knapsack Problem
 - TSP- Travelling Salesman Problem
- **Backtracking**
 - N-queens Problem
 - Sum of Subsets
 - **Graph Coloring**
- **Branch and Bound**
 - LIFO and FIFO Methods
 - Job Selection Problem
 - 0/1 Knapsack Problem



CBS3003- Design and Analysis of Algorithms- References

Text Books:

1. Horowitz, E., Sahni, S., & Rajasekaran, S. Fundamental of computer algorithms, Hyderabad, Universities Press; Second edition, 2008.
2. Kleinberg J, Tardos E. Algorithm design. Pearson Education India; 2006.

Reference Books:

1. Knuth Donald E, “ Art of Computer Programming: Fundamental algorithms”, Third edition, Pearson Publishers, 2011.
2. Pat Morin, “ Open Data Structures: An Introduction(Open Paths to Enriched Learning)”, 31 st Edition, UBC Press, 2013.



BACKTRACKING

- One of the mostly used general design technique which follows a strategy for guessing at a solution and backing up when deadlock situation is reached.
- Many problems which deal with **searching for a set of solutions** or **which ask for an optimal solution satisfying some constraints** can be solved using the backtracking.
- In many applications, the desired solution is expressible as an n-tuple $(x_1, \dots, x_n)$, where the x_i are chosen from some finite set S_i .
- Often the problem to be solved calls for finding one vector that maximizes (or minimizes or satisfies) a criterion function $P(x_1, \dots, x_n)$



BACKTRACKING

- **Explicit and Implicit Constraints**

- Explicit constraints are rules that restrict each X_i to take on values only from a given set.
- Explicit constraints depend on the particular instance “ I ” of the problem being solved. All tuples that satisfy the explicit constraints define a possible solution space for “ I ”.
- Implicit constraints are rules that determine which of the tuples in the solution space of “ I ” satisfy the criterion function.

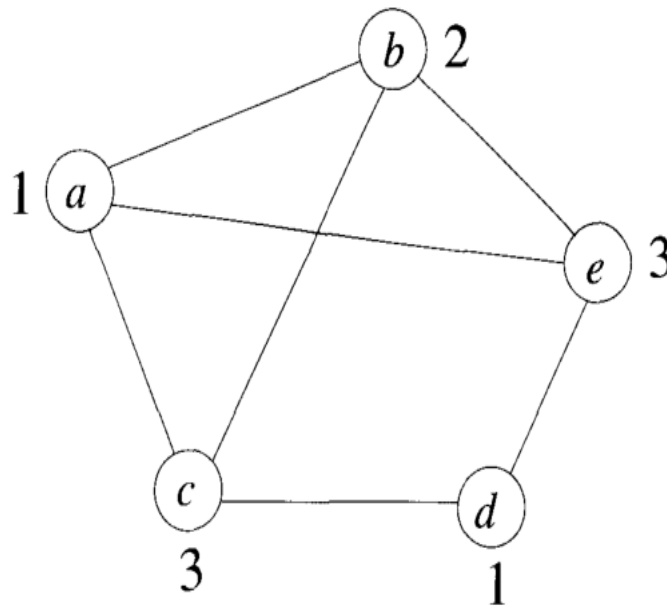


GRAPH COLORING

- Let $G=(V,E)$ be a graph such that 'm' be positive integer.
- **Objective:** To check whether the nodes of Graph "G" can be coloured in such a way such that no two adjacent nodes have the same colour using "m" number of colours.
- Also termed as **M- Colourability Decision Problem**



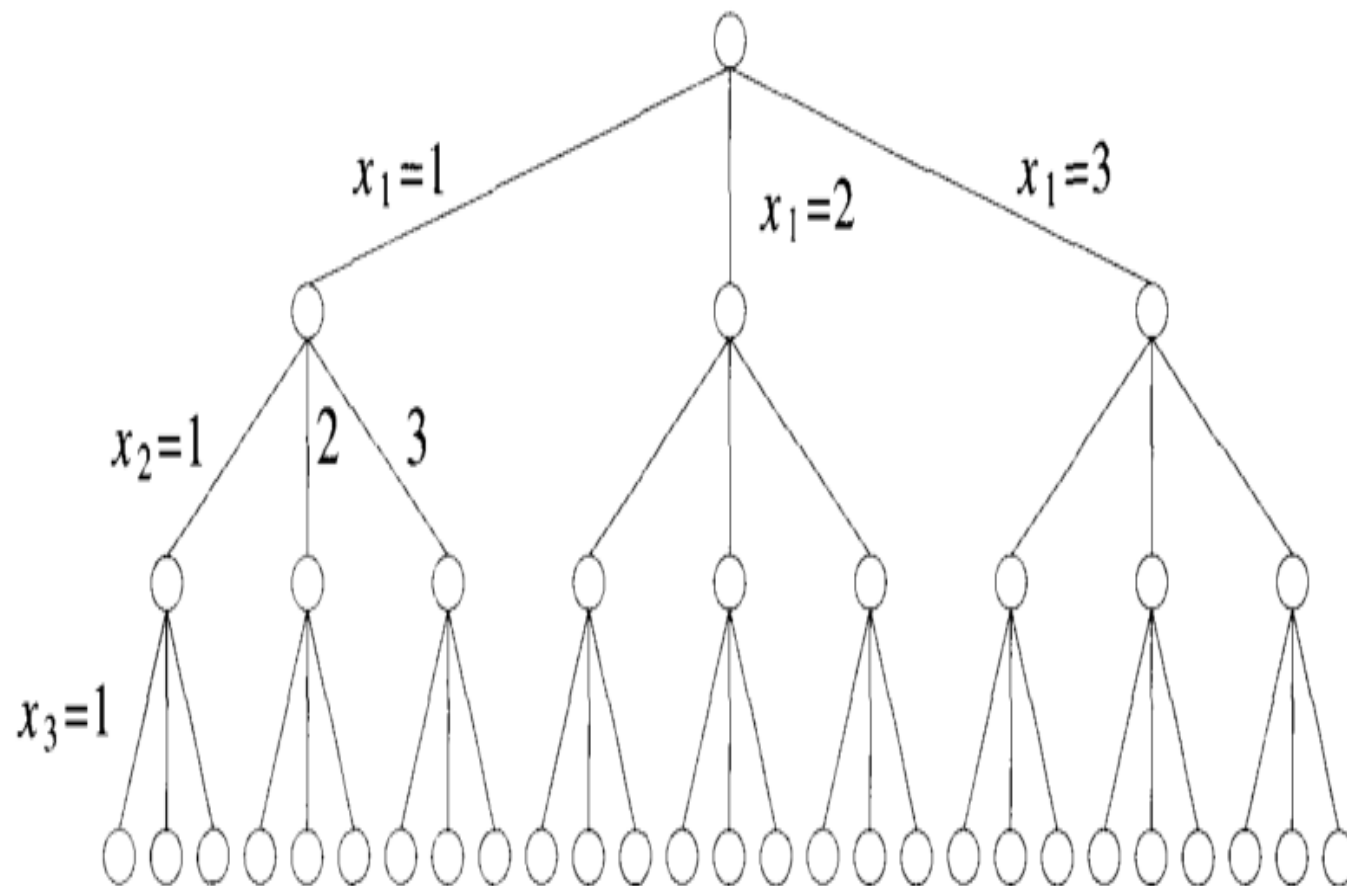
GRAPH COLORING- Example



Chromatic Number of the Graph: Smallest integer “m” in which the Graph G can be coloured



State Space Tree for m colouring when $n=3$ and $m=3$



GRAPH COLORING - ALGORITHM

Algorithm mColoring(k)

```
// This algorithm was formed using the recursive backtracking
// schema. The graph is represented by its boolean adjacency
// matrix  $G[1 : n, 1 : n]$ . All assignments of  $1, 2, \dots, m$  to the
// vertices of the graph such that adjacent vertices are
// assigned distinct integers are printed.  $k$  is the index
// of the next vertex to color.
{
```

repeat

 { // Generate all legal assignments for $x[k]$.

 NextValue(k); // Assign to $x[k]$ a legal color.

if ($x[k] = 0$) **then return**; // No new color possible

if ($k = n$) **then** // At most m colors have been
 // used to color the n vertices.

write ($x[1 : n]$);

else mColoring($k + 1$);

} **until** (false);

}



GRAPH COLORING - ALGORITHM

Algorithm NextValue(k)

```
//  $x[1], \dots, x[k-1]$  have been assigned integer values in
// the range  $[1, m]$  such that adjacent vertices have distinct
// integers. A value for  $x[k]$  is determined in the range
//  $[0, m]$ .  $x[k]$  is assigned the next highest numbered color
// while maintaining distinctness from the adjacent vertices
// of vertex  $k$ . If no such color exists, then  $x[k]$  is 0.
{
    repeat
    {
         $x[k] := (x[k] + 1) \bmod (m + 1)$ ; // Next highest color.
        if ( $x[k] = 0$ ) then return; // All colors have been used.
        for  $j := 1$  to  $n$  do
        {
            // Check if this color is
            // distinct from adjacent colors.
            if ( $(G[k, j] \neq 0) \text{ and } (x[k] = x[j])$ )
            // If  $(k, j)$  is an edge and if adj.
            // vertices have the same color.
                then break;
        }
        if ( $j = n + 1$ ) then return; // New color found
    } until (false); // Otherwise try to find another color.
}
```


GRAPH COLORING - WORKING

GRAPH COLORING

$$G = \{V, E\}$$



$M = 3 \text{ colors}$

ADJACENCY MATRIX

	1	2	3
1	0	1	1
2	1	0	1
3	1	1	0

O/P:

$x[1]$	0	1	
$x[2]$	0	1	2
$x[3]$	0	1	3

Alg Mcoloring[i]

{ repeat

Next value(i).

if ($x[i] = 0$)
 $i = 0$ - False

if ($k = 1$) - False

else Mcoloring[i+1]

Alg Next Value(i)

{ repeat

$x[i] = (0 + 1) \text{ Mod}(4)$

if ($x[i] = 0$) then return

for $j = 1$ to n do

$j = 1, 2, 3$

if ($0 \neq 0$)

$j = 1$ if ($G[i][1] \neq 0$) & &
 $x[i] = x[1]$

$j = 2$ if ($G[i][2] \neq 0$) & &
 $x[i] = x[2]$

$j = 3$ if ($G[i][3] \neq 0$) & &
 $x[i] = x[3]$

if ($j = m + 1$) return

Next page (2)

GRAPH COLORING - WORKING

Alg M coloring (2).

{
repeat
{

Next value (2)

if $(X[2] = 0)$ $\leftarrow$ False.
 $2 = 0$

if $(k = n)$ - false.
 $2 = 3$

else M coloring (2+1)

Next value (2)

{
repeat
{

$X[2] = (1 \text{ Mod } 4) = 1$

if $(X[2] = 0)$ - false
 $1 = 0$

for $j = 1$ to n do

$j = 1, 2, 3$

if $(G[2,1] \neq 0 \text{ and } X[2] = X[1])$
 $1 \neq 0$ $1 = 1$

then Break;

if $(i = 4)$ False.

Until (False) - So Try to find another color.

Repeat the loop for finding next color.

repeat
{

$X[2] = 2 \text{ Mod } 4 = 2$

if $(2 = 0)$ False.

for $j = 1$ to n do
 $j = 1, 2, 3$

if $(G[2,1] \neq 0 \text{ and } X[2] = X[1])$
 $1 \neq 0$ $2 = 1$
 True False

$j = 2$
 $0 \neq 0$
if $(G[2,2] \neq 0 \text{ and } X[2] = X[2])$
 False $2 = 2$
 True

$j = 3$
if $(G[2,3] \neq 0 \text{ and } X[2] = X[3])$
 $1 \neq 0$ $2 = 0$
 True False

if $(4 = 4)$ action // New color found.

Next page (3)

GRAPH COLORING - WORKING

Alg M colouring (3)

{ repeat

Next Value (3)

if $(x[3] = 0)$
3 = 0 false

if $(k = n)$
3 = 3 True

WRITE $[X[1:n]]$

$x[1]$	$x[2]$	$x[3]$
1	2	3

}

Next Value (3)

{ repeat

$x[3] = 1 \text{ Mod } 4 = 1$

if $(x[3] = 0)$ - False

for $j = 1$ to n do

$j = 1, 2, 3$

$j = 1$

if $(G[3,1] \neq 0 \text{ and } x[3] = x[1])$
1 $\neq$ 0 1 = 1

3

Break

if $(i = 4)$ - false.

} - Repeat the Loop for next color

repeat

$x[3] = 2 \text{ Mod } 4 = 2$

if $(2 = 0)$ false.

for $j = 1$ to n do

$j = 1, 2, 3$

$j = 1$

if $(G[3,1] \neq 0 \text{ and } x[3] = x[1])$

2 = 1

$j = 2$

if $(G[3,2] \neq 0 \text{ and } x[3] = x[2])$

2 = 2

Break

}

if $(2 = 4)$ - false

} - Repeat the Loop for next color.

GRAPH COLORING - WORKING

④

repeat
 $\{$
 $X[3] = 3 \text{ Mod } 4 = 3$
 $\text{if } (3 = 0) \text{ False}$
 $\text{for } (j = 1 \text{ to } m) \text{ do}$
 $\{$
 $j = 1, 2, 3.$
 $j = 1$
 $\text{if } ((G[3,1] \neq 0 \ \&\& \ X[3] = X[1]))$
 $\quad \text{True} \quad \quad \quad 3 = 1$
 $\quad \quad \quad \text{False}$
 $j = 2$
 $\text{if } ((G[3,2] \neq 0 \ \&\& \ X[3] = X[2]))$
 $\quad \text{True} \quad \quad \quad 3 = 2$
 $\quad \quad \quad \text{False}$
 $j = 3$
 $\text{if } ((G[3,3] \neq 0 \ \&\& \ X[3] = X[3]))$
 $\quad \text{False} \quad \quad \quad 3 = 3$
 $\quad \quad \quad \text{True}$
 $\}$
 $\text{if } (4 = 3 + 1) - \text{New color found}$
 $\leftarrow$
 Return to the Call.
 i.e. (Page 3)



Thank You

